

Returning to clustering, we still don't have a formalized structure for clustering. The attempts were numerous:

- KEELINBERG (2003): AXIOMATIZATION OF CLUSTERING FUNCTION

$$F((X, d)) = (C_1, \dots, C_k)$$

|| (DESIRABLE) PROPERTIES OF F :

- SCALE INVARIANCE: F returns the same partition after scaling the data
- RICHNESS: $\forall C_i \in (C_1, \dots, C_k) \exists d \text{ s.t. } F((X, d)) = (C_1, \dots, C_k)$
(so, distance function DETERMINES the output)
- CONSISTENCY: similar points will be clustered together and dissimilar points will be separated

We can prove an impossibility result:

IMPOSSIBILITY RESULT

$\nexists$ clustering function F that satisfies at the same time SCALE INVARIANCE, RICHNESS AND CONSISTENCY.

NEURAL NETWORKS

Initially, we approached to the optimization problem parametrically by minimizing the expected risk (or loss function).

$$f: X \rightarrow Y \quad S' = \{(x_1, t_1), \dots, (x_m, t_m)\} \quad t_i = f(x_i)$$

$$H \quad h(x) = h(x; \vartheta) = w_1 x_1 + w_2 x_2 + w_0 \quad \vartheta = \{w_1, w_2, w_0\}$$

$L(h_S)$ is the loss function or ERM

Now, we will proceed by a parametrical approach as above, but $h(x)$ will not be a NEURAL NETWORK FUNCTION

FEED-FORWARD NEURAL NETWORKS (FNN)

OR

ARTIFICIAL NEURAL NETWORKS (ANN)

OR

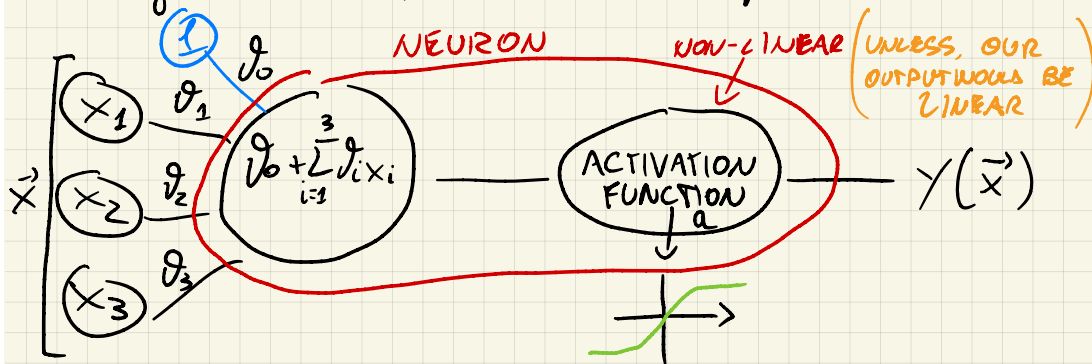
MULTI-LAYER PERCEPTRON

In previous models, it wasn't possible to fix or edit some variables during training (kernel SVM could, but with a lot of limitations). Here we will deal with more powerful but more complex models.

SHALLOW FNN

($\neq$ DEEP)

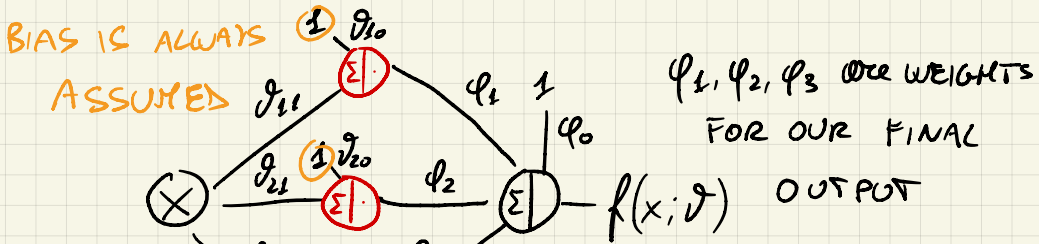
The main element of NNs are neurons (human-based working elements). The model consists of two units:



x_1, x_2, x_3 are input parameters, but 1 is the BIAS.

$$y(x_1, x_2, x_3) = a\left(\theta_0 + \sum_{i=1}^3 \theta_i x_i\right)$$

Now, let's build a neural network, obtained by connecting neurons.

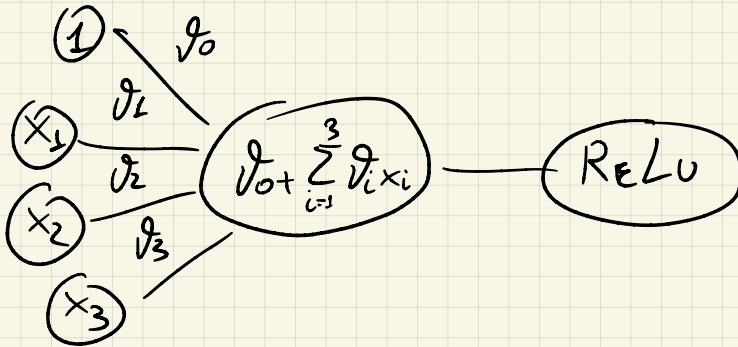


$$\theta = \{v_{10}, v_{11}, v_{20}, v_{21}, v_{30}, v_{31}, \varphi_1, \varphi_2, \varphi_3, \varphi_0\}$$

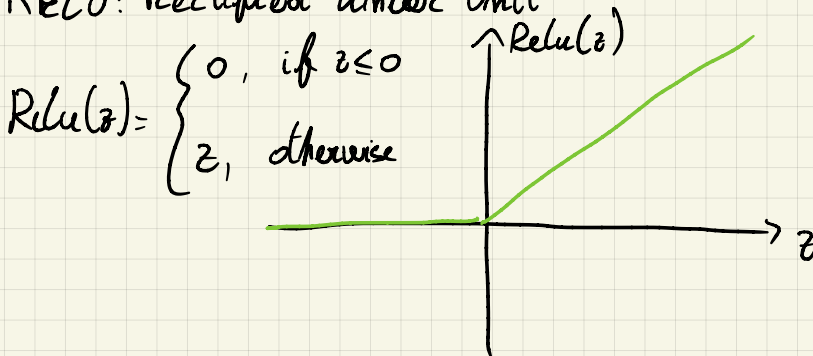
$$f(x_i; \theta) = \varphi_0 + \varphi_1 \cdot a(v_{10} + v_{11}x) + \varphi_2 \cdot a(v_{20} + v_{21}x) + \varphi_3 \cdot a(v_{30} + v_{31}x)$$

FOR NOW, ACTIVATION FUNCTION IS UNIFORM FOR ALL NEURONS

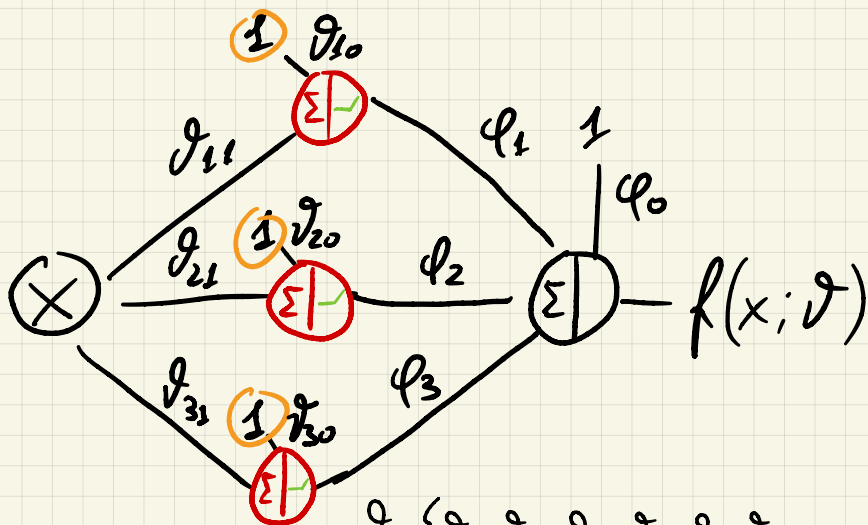
Now, let's see what happens at the activation function (different from the sigmoid, but still non-linear):



ReLU: Rectified Linear Unit



Let's our NN with RELU:



$$\mathcal{V} = \{v_{10}, v_{11}, v_{20}, v_{21}, v_{30}, v_{31}, p_1, p_2, p_3, p_0\}$$

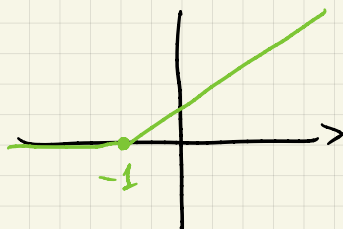
$$f(x_i; \sigma) = \varphi_0 + \varphi_1 \cdot a(\vartheta_{10} + \vartheta_{11}x) + \varphi_2 \cdot a(\vartheta_{20} + \vartheta_{21}x) + \varphi_3 \cdot a(\vartheta_{30} + \vartheta_{31}x)$$

let's apply $y = \text{Relu}(w_0 + w_1 x)$ and see some cases:

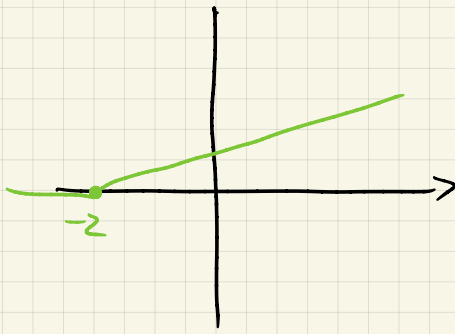
- $\mathcal{V}_{10} = 0; \mathcal{V}_{11} = 1$; $f(x; \mathcal{V}) = \text{ReLU}(x)$



- $\mathcal{I}_{10} = 1; \mathcal{I}_{11} = 1$; $f(x; \mathcal{I}) = \text{ReLU}(1+x)$



• $\vartheta_{10} = 1; \vartheta_{11} = \frac{1}{2}$; $f(x; \vartheta) = \text{ReLU} \left(1 + \frac{1}{2}x \right)$

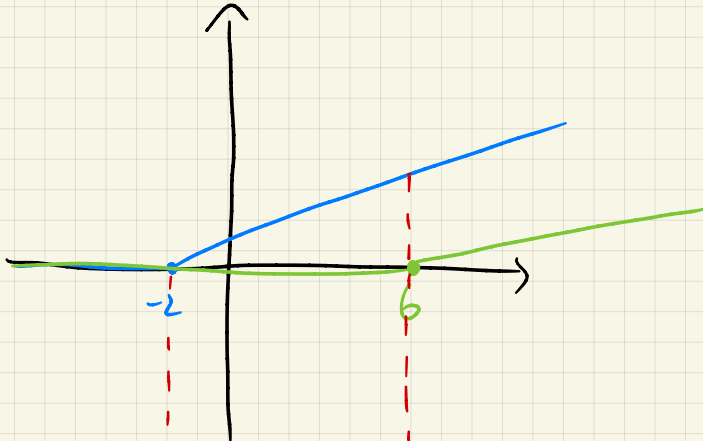


What if I combine the first 2 neurons' parameters?

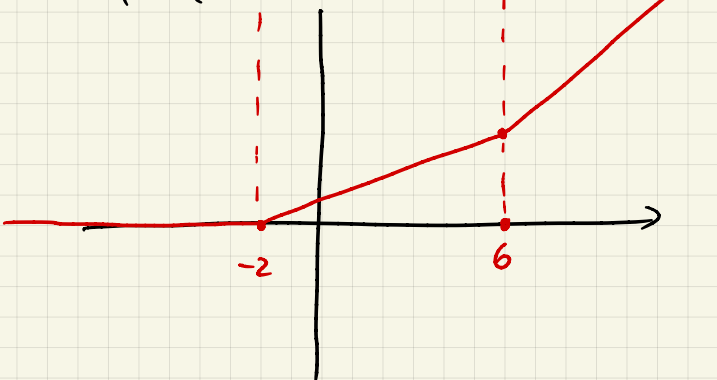
• $\mathcal{I}_{10} = 1; \mathcal{I}_{11} = \frac{1}{2} \rightarrow \underline{\text{Relu}(1 + \frac{x}{2})}$

• $\mathcal{I}_{20} = -2; \mathcal{I}_{21} = \frac{1}{3} \rightarrow \underline{\text{Relu}(-2 + \frac{x}{3})}$

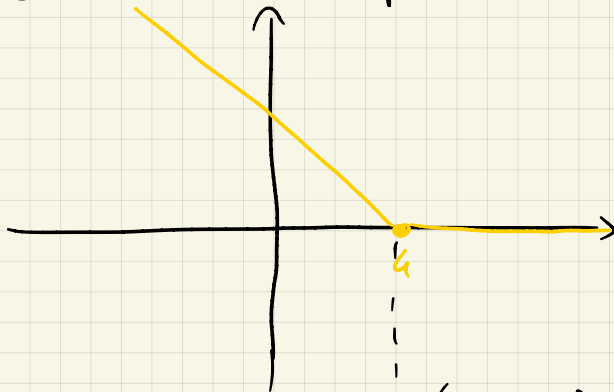
WE STILL DO NOT
CONSIDER THE BIAS



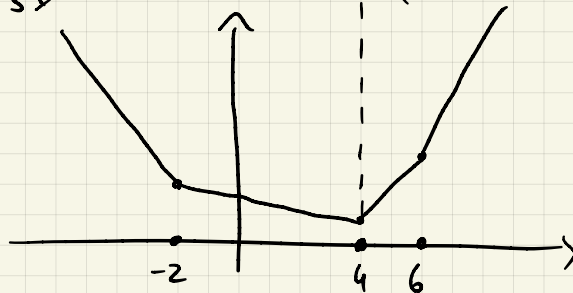
For $\varphi_0 = 0, \varphi_1 = \varphi_2 = 1$:



Let's see what happens also with the 3rd neuron, in such a way that I want to produce this function:

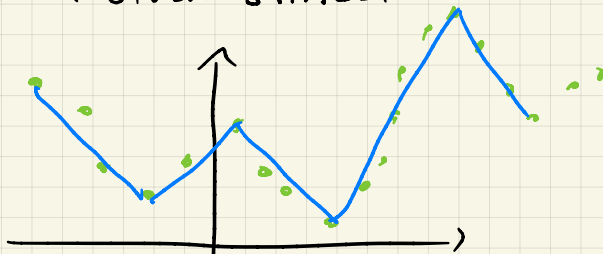


• $v_{30}=8$; $v_{31}=-2 \rightarrow \text{ReLU}(8-2x)$



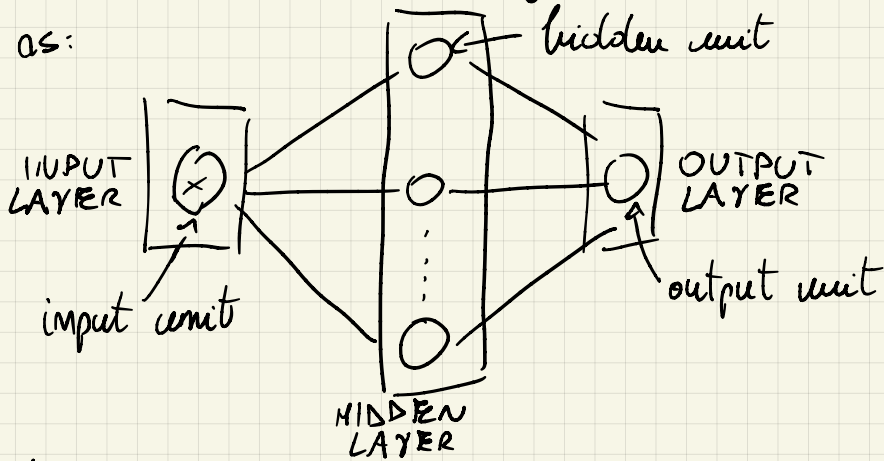
So, by "tuning" the parameters, we build a piecewise linear function by using a non linear one as input. But let's go back to our goal: we want to minimize our loss function for a given dataset.

EXAMPLE FOR A GIVEN DATASET



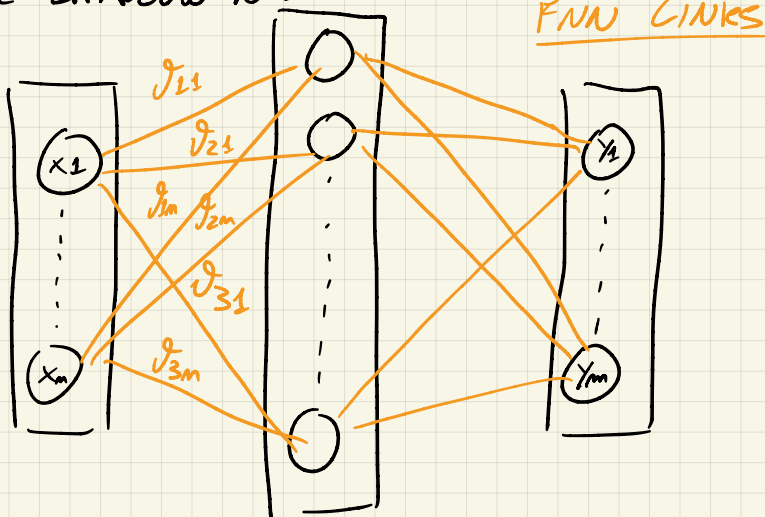
So, the intuition is that for a feature map, by building a number of units I can fit very well on our target.

In our neural network build, our units are labelled as:



An input and an output layer could have multiple units.
 In a SHALLOW FNN, there is JUST ONE HIDDEN LAYER.
 Let's generalize it.

GENERIC SHALLOW NN



In SHALLOW FNN, every unit in a layer has a link to every unit of the next layer in the model.

The function will be:

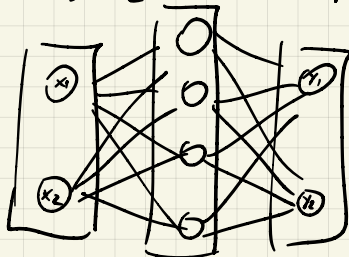
$$f(x_1, \dots, x_m; \theta) = \langle y_1, \dots, y_m \rangle$$

Let's see an example with a function that, given two points, it gives middle distance between them on a line and their total distance:



$$\mathbb{R}^2 \rightarrow \mathbb{R}^2$$

$$f(x_1, x_2) = \langle y_1, y_2 \rangle$$



So, how powerful this FNN are?

UNIVERSAL APPROXIMATION THEOREM

A shallow FNN with a linear output layer s.t. the hidden units are "squashing*" can approximate any Borel-measurable function, up to an arbitrary error ϵ , provided it has a sufficient number of hidden units.

* squashing = constraining

This theorem says that you can always come up with a result up to a desired error if it is used the right number of units.

SQUASHING FUNCTION CONSTRAIN THE OUTPUT TO SOME INTERVAL:

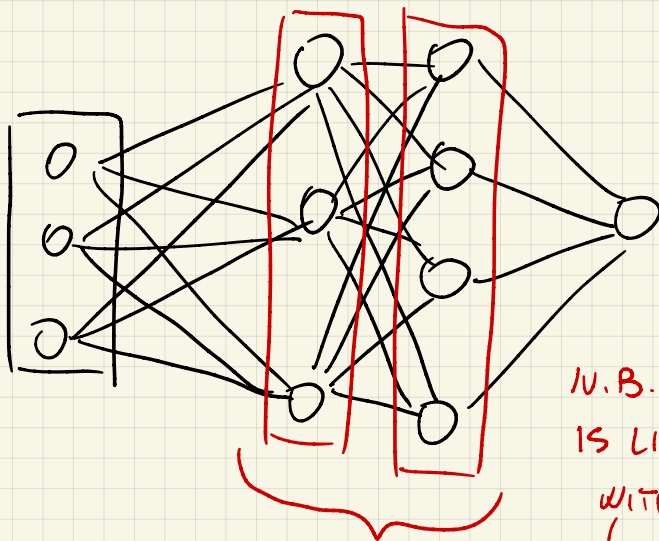
$$g: X \rightarrow (a, b)$$

So, a SHALLOW NETWORK is built by a single hidden layer. Let's try to expand our hidden layer. First, let's define two terms:

- CAPACITY: # of hidden units
- WIDTH (of a layer): # of units in the layer

Keep in mind that DIFFERENT HIDDEN LAYERS can have DIFFERENT CAPACITY.

When a network has more than a single hidden layer, we talk about DEEP NETWORKS:



> 1 HIDDEN LAYERS



DEEP

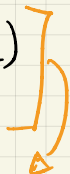
We know that a SHALLOW NN can approximate any function, so why do we need a DEEP NETWORK?

First, DEEP NETWORKS can have more linear regions. Also, for some mysterious reasons, they are faster to train and generalize better on unseen examples.

So, deep networks are relatively better, but are very hard to understand and to use.

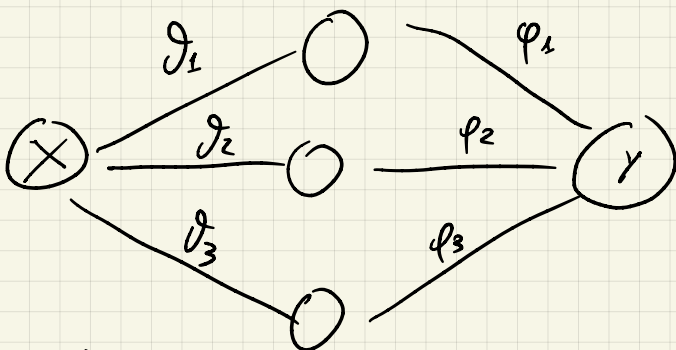
NEURAL NETWORK STRUCTURE

- WIDTH
 - DEPTH
- THERE ARE NO GUIDELINES ON HOW MUCH WIDTH AND DEPTH YOU NEED
- ACTIVATION FUNCTIONS (HIDDEN LAYER, OUTPUT LAYER)
 - LOSS FUNCTION



THESE ARE DEEPLY PROBLEM-RELATED

EXAMPLE



$$y = h(x; \vartheta)$$

In the models we have seen so far, for a given x , y is an approximation (or prediction) of $f(x)$:

$$y = h(x) \sim f(x), \quad f: \mathbb{R} \rightarrow \mathbb{R}$$

the same happens for classification, but with probability:

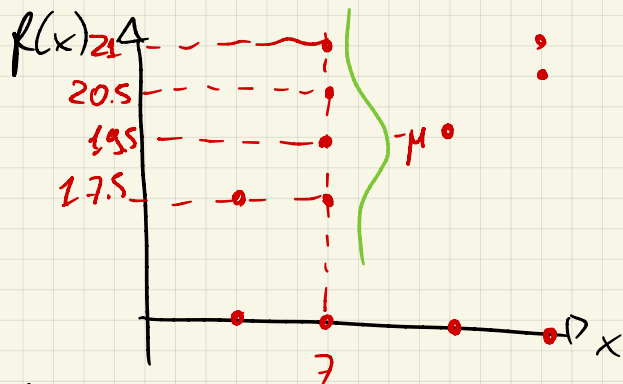
$$f: \mathbb{R}^3 \rightarrow \{0, 1\}$$

$$y = h((x_1, x_2, x_3)) \sim f(x) \in \{0, 1\}$$

$$h((x_1, x_2, x_3)) = \text{probability}$$

With neural networks, for a given input, the output will be a probability distribution of the possible output.

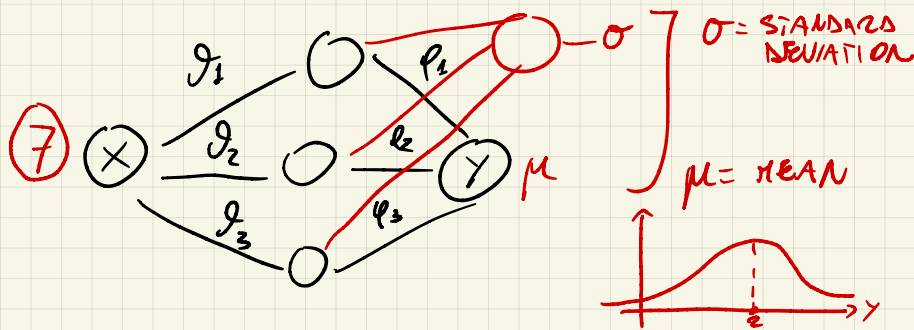
For example, let's see a very noisy dataset, where for a given x in a specific time the temperature measurement has been measured different for 10 times:



Let's imagine that we want that our function retrieves a distribution of the temperature values. So I'm just computing the mean of a Gaussian distribution.

On a neural network, although, we retrieve the probability distribution of those values.

In example, let's assume our network has given a conditional probability distribution as output.



So, our network will retrieve a mean with a standard deviation from the mean.

So here the difference is that instead of studying the most probable output from a distribution, here I am retrieving the probability distribution as output of the network.

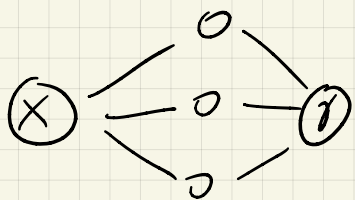
In example, for a binary classification, we are retrieving the probability distribution of the classification output between the whole dataset.

$$p(t|\lambda) = \begin{cases} 1-\lambda, & y=0 \\ \lambda, & y=1 \end{cases}$$

It is a BERNULLI DISTRIBUTION that could be written as:

$$p(t|\lambda) = (1-\lambda)^{1-t} \cdot \lambda^t \quad \text{BERNULLI DISTRIBUTION}$$

The output parameter of the network, γ , will be interpreted as λ :



$$\gamma(7) = 0.62 = \lambda$$

$$p(1|\gamma(7)) = (1-0.62)^0 \cdot 0.62^1 = 0.62$$

$$p(0|\gamma(7)) = (1-0.62)^1 \cdot 0.62^0 = 1-0.62$$

So, the network predicted the probability of x to be 1. Then, I can analyze which is the probability of this prediction using a conditional distribution.

Back to our problem, my goal is to maximise the fitting of the prediction/classification. So for a given training set S :

$$S = \{(x_1, t_1), (x_2, t_2), \dots, (x_m, t_m)\}$$

I want to maximise the probability of x_i being t_i . (we still assume that the distribution is uniform):

$$p(t|x, \theta)$$

$$\hat{\theta} = \operatorname{argmax}_{\theta} \prod_{i=1}^m p(t_i|x_i; \theta) = \operatorname{argmax}_{\theta} \prod_{(x,t) \in S} p(t|x; \theta)$$

MAXIMUM LIKELIHOOD PRINCIPLE

N.B.: our $\hat{\theta}$ can be tiny due to APPROXIMATION ERROR (for a large scale of parameters)

Also, $\hat{\theta}$ can be exploited this way (to find log likelihood):

$$\hat{\theta} = \operatorname{argmax}_{\theta} \log \left(\prod_{(x,t) \in S} (p(t|x; \theta)) \right) =$$

$$= \operatorname{argmin}_{\theta} -\log \left(\prod_{(x,t) \in S} (p(t|x; \theta)) \right) =$$

$$= \operatorname{argmin}_{\theta} \underbrace{\sum_{(x,t) \in S} -\log(p(t|x; \theta))}_{\text{LOSS FUNCTION } J(\theta)} = \operatorname{argmin}_{\theta} J(\theta)$$

LOSS FUNCTION $J(\theta)$
(NEGATIVE LOG LIKELIHOOD)

This is the loss function because it is what we want to minimise to fit the data.

let's now see the general approach to design the loss function, in steps:

1. FIX PARAMETRIZED DISTRIBUTION OVER PREDICTIONS
 $p(t|\lambda)$

2. SET THE MODEL (THE NN) TO PREDICT (SOME) PARAMETERS OF THE DISTRIBUTION

$h(x; \theta)$ NETWORK OUTPUT
if $h(x; \theta)$ predicts $p(t|\lambda)$, then:

$$p(t|\lambda) = p(t|x; \theta)$$

3. TRAIN THE MODEL TO MINIMIZE $J(\theta)$ OVER THE TRAINING SET

$$\hat{\theta} = \underset{\theta}{\operatorname{argmin}} J(\theta)$$

4. GIVEN (SOME) x , PREDICT THE OUTPUT BASED ON:

$$y(x) = \underset{t}{\operatorname{argmax}} p(t|x; \hat{\theta}) \quad \text{VALUE THAT MAXIMIZES } t$$

let's now see for every type of optimization model our NN work:

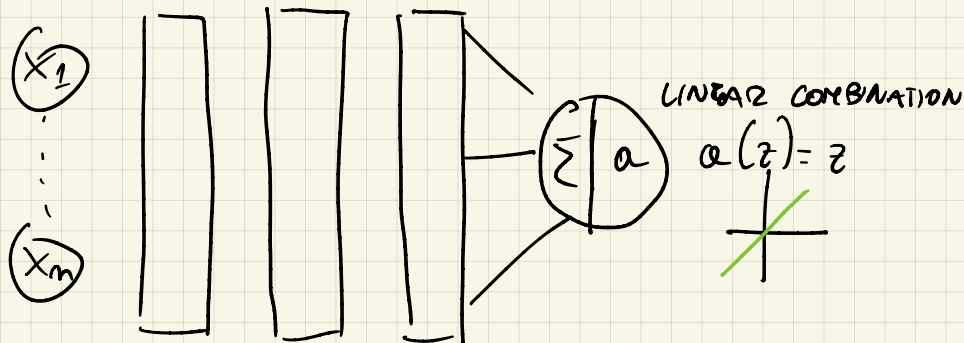
REGRESSION
 $f: \mathbb{R}^m \rightarrow \mathbb{R}$

$$\Sigma = \begin{bmatrix} \sigma_1 & & & 0 \\ & \sigma_2 & & \\ & & \ddots & \\ 0 & & & \sigma_m \end{bmatrix}$$

1. FIX DISTRIBUTION: $p(t|\mu, \sigma)$ " $p(t|\mu, \Sigma)$ IN A MULTI-VARIANT CASE
NORMAL DISTRIBUTION $\mathcal{N}(\mu, \sigma)$

2. DESIGN NETWORK TO PREDICT DISTRIBUTION PARAMETER(S).

In this case, we predict just the mean (μ).



$$\mu = h(\vec{x}; \vartheta) ; p(t | \mu, \sigma) = p(t | h(\vec{x}, \vartheta), \sigma) = N(h(\vec{x}; \vartheta), \sigma)$$

σ is unknown, but we don't need it for a maximization problem

3. MODEL TRAINING

$$J(\vartheta) = \sum_{(x,t) \in S} -\log N(t | h(\vec{x}; \vartheta), \sigma)$$

This is exactly what we did with linear regression. Indeed:

$$J(\vartheta) = \sum_{(x,t) \in S} \|h(x; \vartheta) - t\|^2$$

$$\underset{\vartheta}{\operatorname{argmin}} J(\vartheta) = \underset{\vartheta}{\operatorname{argmin}} J'(\vartheta)$$

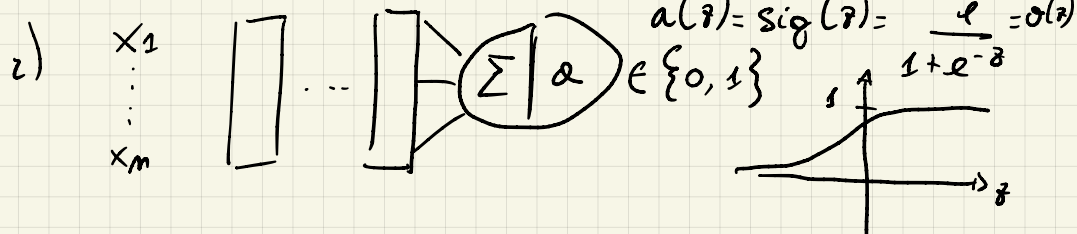
PROOF ON THE BOOK

What changes if I have, for m inputs, m outputs? I will just have a distribution over a vector ($\mu \rightarrow \vec{\mu}, \sigma \rightarrow \vec{\sigma}$), where every unit is still linear (i.e., every unit has $a(z) = z$). While training, our expected t will be a vector of expected values and:

$$J'(\vartheta) = \sum \sum \|h(x; \vartheta) - t\|^2$$

BINARY CLASSIFICATION

$$1) p(t | \lambda) = (1 - \lambda)^{1-t} \cdot \lambda^t$$



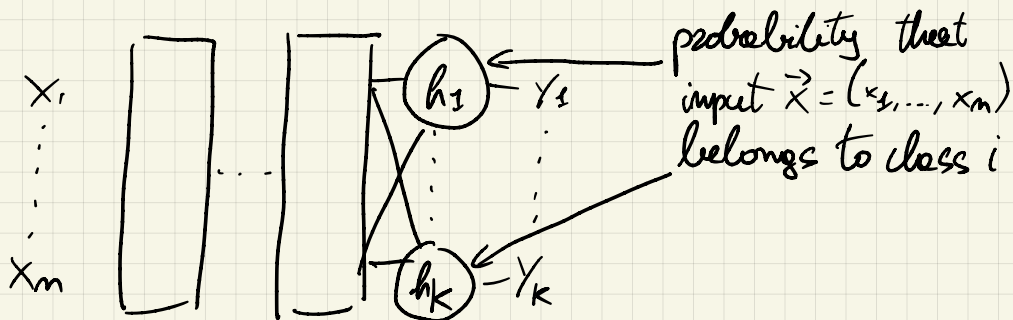
$$p(t | h(x; \theta)) = p(t | x) = (1 - \sigma(h(x; \theta)))^{1-t} \cdot \sigma(h(x; \theta))^t$$

$$3) J(\theta) = -\log \prod_{(x,t) \in S} p(t | x) = \sum_{(x,t) \in S} \left[-(1-t) \cdot \log(1 - \sigma(h(x; \theta))) - t \cdot \log(\sigma(h(x; \theta))) \right]$$

BINARY CROSS-ENTROPY

MULTI-CLASS CLASSIFICATION

Here, we will see also the network architecture build.
 let's have k classes: K -classes



λ_i : probability that input belongs to class i

$$p(y=k) = \lambda_k$$

To fix the outputs, we will pass them to a softmax layer:

$$y_i = \text{softmax}_i(h) = \frac{e^{h_i}}{\sum_{j=1}^K e^{h_j}}$$

$$p(y=k|x) = \text{softmax}_k(h(x; \theta))$$

$$\begin{aligned} J(\theta) &= - \sum_{(x, t) \in S} \log(\text{softmax}_t(h(x; \theta))) = && \text{MULTI-CLASS} \\ &= - \sum_{(x, t) \in S} h_t(x; \theta) - \log \sum_{j=1}^K e^{h_j(x; \theta)} && \text{CROSS-ENTROPY} \end{aligned}$$

Using exponentials for softmax normalization AMPLIFIES THE DIFFERENCES between all classes (you could use also other operations/functions)

N.B.: each output layer here DEPENDS ON EACH OTHER

To resume:

1. REGRESSION

- OUTPUT: LINEAR
- LOSS: SUM OF SQUARED ERRORS

2. BINARY CLASSIFICATION

- OUTPUT: SIGMOID $\rightarrow$ IT'S A SQUASHING FUNCTION!
- LOSS: BINARY CROSS-ENTROPY

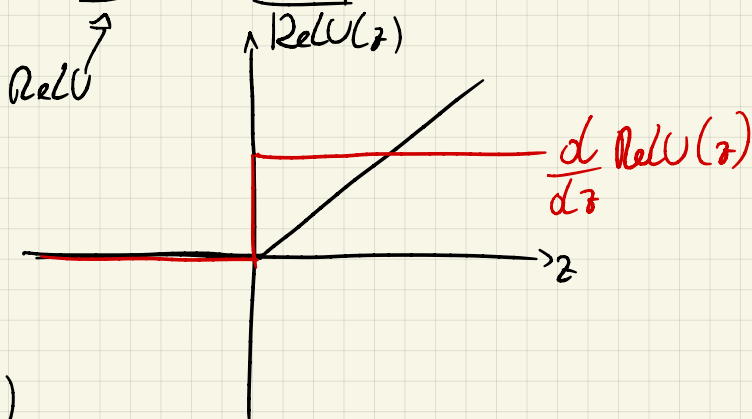
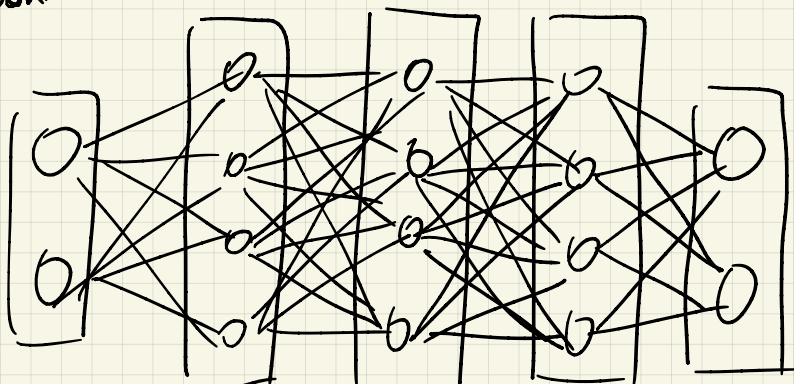
3. MULTICLASS CLASSIFICATION

- OUTPUT: SOFTMAX
- LOSS: MULTICLASS CROSS-ENTROPY

What remains to discuss are the ACTIVATION FUNCTIONS for hidden units.

REMINDER: we are using FULLY CONNECTED NEURAL NETWORKS
Basically, activation functions depend by the specific architecture we are dealing with.

let's see an example while applying $\text{Relu}(x)$ as an activation function:



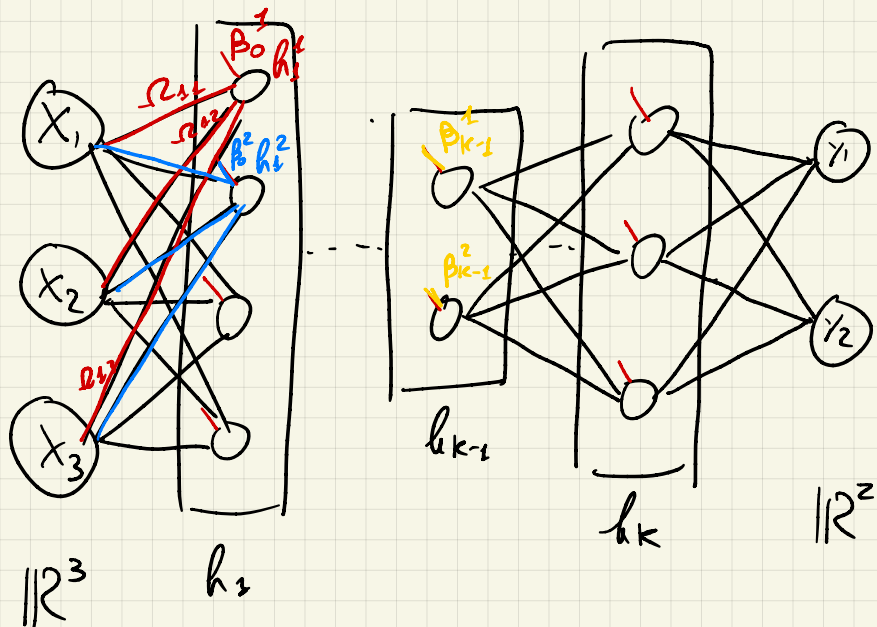
$$y = f(\vec{z}; \vec{\theta})$$

N.B.: ACTIVATION FUNCTIONS MUST BE DIFFERENTIABLE
So, in this case we will just apply GRADIENT DESCENT
in order to minimize the loss function.

In general, we used graphs to represent NN. let's see
a more convenient representation:



MATRIX NOTATION FOR NN



a : activation function for hidden layers

$$h_1^1 = a[\Omega_{11}x_1 + \Omega_{12}x_2 + \Omega_{13}x_3 + \beta_0^1] =$$

$$= a\left([\Omega_{11} \ \Omega_{12} \ \Omega_{13}] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \beta_0^1\right)$$

$$h_1^2 = a[\Omega_{21}x_1 + \Omega_{22}x_2 + \Omega_{23}x_3 + \beta_0^2] =$$

$$= a\left([\Omega_{21} \ \Omega_{22} \ \Omega_{23}] \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \beta_0^2\right)$$

Then put together h_1^1 and h_1^2 :

$$\begin{bmatrix} h_1^1 \\ h_1^2 \\ h_1^3 \end{bmatrix} = a \left(\begin{bmatrix} -R_{11} & -R_{12} & -R_{13} \\ -R_{21} & -R_{22} & -R_{23} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix} + \begin{bmatrix} \beta_0^1 \\ \beta_0^2 \end{bmatrix} \right) = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix}$$

ELEMENT-WISE (YOU APPLY a TO)
APPLICATION (EVERY ELEMENT)

So we can generalize the notation as:

$$\underbrace{\begin{bmatrix} h_1^1 \\ h_1^2 \\ h_1^3 \\ h_1^4 \end{bmatrix}}_{h_1} = a \left(\underbrace{\begin{bmatrix} -R_{11} & -R_{12} & -R_{13} \\ \vdots & \vdots & \vdots \\ -R_{41} & \dots & -R_{43} \end{bmatrix}}_{R_0} \underbrace{\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}}_{\vec{x}} + \underbrace{\begin{bmatrix} \beta_0^1 \\ \vdots \\ \beta_0^4 \end{bmatrix}}_{\beta_0} \right)$$

$$h_1 = a(-R_0 \vec{x} + \vec{\beta}_0) = a(-R_0 x + \beta_0)$$

So, for the other layers:

$$h_2 = a(-R_1 h_1 + \beta_1)$$

$$h_{k-1} = a(-R_{k-2} h_{k-2} + \beta_{k-2})$$

$$h_k = a(-R_{k-1} h_{k-1} + \beta_{k-1})$$

$$y = \textcircled{a}(-R_k h_k + \beta_k) \rightarrow \text{OUTPUT ACTIVATION FUNCTION}$$

The OUTPUT ACTIVATION FUNCTION is the act. function applied in the output units. Based on which output we want, that function could be a sigmoid, a softmax, ecc. as we discussed before.

TRAINING AN MODEL (FITTING)

Our objective is minimizing the loss function. let's see it for the case of linear regression:

$$\hat{\phi} = \underset{\phi}{\operatorname{argmin}} [L(\phi)]$$

$$f(x; \Phi) = \phi_0 + \phi_1 x_1 + \phi_2 x_2 + \phi_3 x_3$$

$$L(\phi) = \sum_{(x,y) \in S} (f(x; \phi) - y)^2$$

OUR SET IS FIXED, SO WE ONLY NEED TO CHANGE THE PARAMETERS S.T. $L(\phi)$ IS MINIMIZED

$$\frac{\partial L}{\partial \phi} = \begin{bmatrix} \frac{\partial L}{\partial \phi_0} \\ \frac{\partial L}{\partial \phi_1} \\ \vdots \\ \frac{\partial L}{\partial \phi_3} \end{bmatrix} = \begin{bmatrix} \frac{\partial L(\bar{\phi})}{\partial \phi_0} \\ \vdots \\ \frac{\partial L(\bar{\phi})}{\partial \phi_3} \end{bmatrix}$$

N.B.: THIS FORMULA IS WRITTEN FOR RANDOM $\phi_0, \phi_1, \phi_2, \phi_3$ VALUES.
OF COURSE, FOR A GIVEN PARAMETER WE USE THE CURRENT VALUE OF THE PARAMETER ($\bar{\phi} = (\bar{\phi}_0, \bar{\phi}_1, \bar{\phi}_2, \bar{\phi}_3)$)

So, after computing gradient descent, I update my ϕ :

$$\phi \leftarrow \bar{\phi} - \alpha \frac{\partial L(\bar{\phi})}{\partial \phi}$$

↑
LEARNING RATE

Now, if I proceed I could get trapped on a local minimum, but this just depends on how I started from.

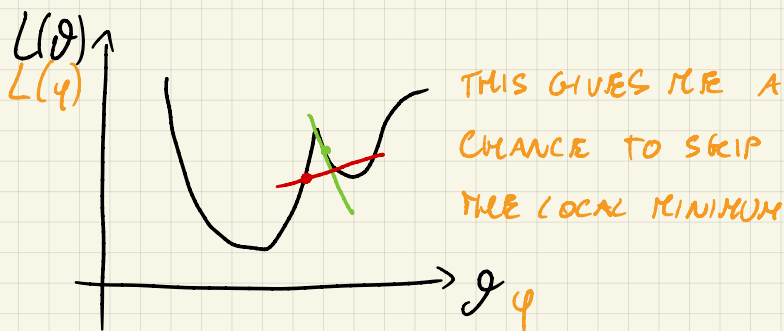
The second problem is that deriving partially is not so easy, because it is needed to apply the gradient descent w.r.t. all the parameters.

Given the set of the best dropter, if i.e. my S is:

$$S = \{(3, 2)\} \rightarrow L((y_1, y_2), (x_1, x_2))$$

To fit our L into S , I have to compute $\frac{\partial L}{\partial \varphi}$, but we have to do it for the composing functions $\overline{\partial \varphi}$ of all the hidden units that are computing a derivative w.r.t. our parameters.

We try to partially solve the problem by using a slightly different gradient descent by adding SOME NOISE (which is random):



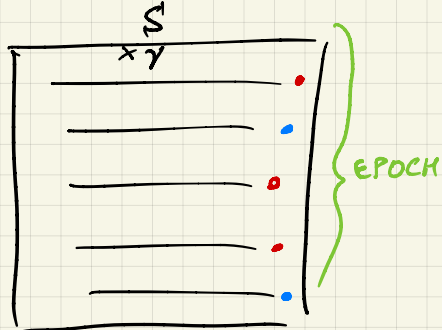
Instead of computing the loss function by using the whole S :

$$L(\varphi) = \sum_{(x,y) \in S} |f(x; \varphi) - y|^2$$

I apply STOCHASTIC GRADIENT DESCENT by randomly mini-batching.

B = mini-batch of S :

$$L(\varphi) = \sum_{(x,y) \in B} |f(x; \varphi) - y|^2$$



When the iteration is over, a new mini-batch is randomly picked w/out the elements of the previous one(s). While the full S has been iterated, we run over an EPOCH. An algorithm can pass through more epochs, as it increases the possibility to find the global minimum.

MOMENTUM

At the next iteration, I compute the gradient this way:

$$\text{MOMENTUM } m_{t+1} \leftarrow \beta \cdot m_t + (1 - \beta) \cdot \sum_B \frac{\partial L(\varphi_t)}{\partial \varphi}$$
$$\varphi_{t+1} \leftarrow \varphi_t - \alpha m_{t+1}$$

φ_t : state of parameters at iteration t

$\beta \in [0, 1)$

If $\beta = 0$, m_{t+1} is just our gradient descent. If it's not, I am combining the gradient I have previously computed with the more previous ones, where the greater is β , the more importance the previous ones have for $t+1$. This is done batch by batch.